

in Wagtail:

```
class BlogPage(Page):
    date = models.DateField("Erstellungsdatum")
    intro = models.CharField("Kurzbeschreibung",
max_length=250)
    body = RichTextField(blank=True)
    category = ParentalManyToManyField('blog.Kategorie',
blank=True)
    tags = ClusterTaggableManager(through=BlogPageTag,
blank=True)

Show more ▾
```

In Wagtail kommen die Feldbezeichnungen normalerweise aus dem Django-Model-Feld. Deshalb funktioniert es bei `date` und `intro`, weil du dort das `verbose_name` als ersten Parameter angegeben hast:

Python

Für die anderen Felder kannst du das genauso machen:

```
body = RichTextField("Inhalt", blank=True)

category = ParentalManyToManyField(
    'blog.Kategorie',
    verbose_name="Kategorien",
    blank=True
)

tags = ClusterTaggableManager(
    verbose_name="Schlagwörter",

Python
)
```

Falls du die Beschriftungen **nur im Wagtail-Admin** ändern möchtest, ohne das eigentliche Modell zu ändern, kannst du das über die Panels machen:

```
from wagtail.admin.panels import FieldPanel

content_panels = Page.content_panels + [
    FieldPanel("date"),
    FieldPanel("intro"),
    FieldPanel("body", heading="Inhalt"),
```

Python

Der Unterschied:

- `verbose_name` → gilt generell für das Modell (Admin, Forms, ModelForms usw.).
- `heading` im `FieldPanel` → überschreibt die Anzeige nur im Wagtail-Admin.

Wenn du mir deine aktuelle `content_panels` -Definition zeigst, kann ich dir sagen, welche Variante in deinem Fall sinnvoller ist.



ich bekomme den Fehler 'navigation_tags' is not a registered tag library. Must be one of: admin_list admin_modify admin_urls auth cache i18n l10n log static tz wagtail_cache wagtailadmin_tags wagtailcore_tags wagtailembeds_tags wagtailimages_tags wagtailsettings_tags wagtailuserbar wagtailusers_tags in mysite\mysite\templates\includes\footer.html

Der Fehler bedeutet, dass in deiner `footer.html` vermutlich etwas wie

`django`

steht, Django aber keine Tag-Library mit diesem Namen findet.

Typische Ursachen:

1. Die Datei `navigation_tags.py` existiert nicht

Für ein Custom Template Tag muss es etwa so aussehen:

```
blog/
├── templatetags/
│   ├── __init__.py
│   └── navigation_tags.py
```

In `navigation_tags.py`:

```
from django import template

register = template.Library()
```

Associated template tag

Python

Wichtig: Im Ordner `templatetags` muss eine `__init__.py` liegen.

2. Die App ist nicht in `INSTALLED_APPS`

Prüfe in `settings.py`:

```
INSTALLED_APPS = [
    ...
```

Python

Wenn die App fehlt, werden die Template Tags nicht registriert.

3. Entwicklungsserver nicht neu gestartet

Nach dem Anlegen von `templatetags` oder `navigation_tags.py` einmal stoppen und neu starten:

Bash

4. Du verwendest ein Wagtail-Tutorial

Viele Wagtail-Tutorials haben eine Datei wie:

Python

mit Funktionen wie:

```
@register.inclusion_tag("includes/navigation.html", takes_context=True)
```

Python

Wenn du Code aus einem Tutorial übernommen hast, aber die Datei nicht mitkopiert hast, entsteht genau dieser Fehler.

5. Du brauchst das Tag gar nicht

Schau in `footer.html` nach:

django

Wenn im Template anschließend gar keine Tags daraus verwendet werden, kannst du die Zeile einfach entfernen.

Kannst du den Inhalt von:

django

und die ersten 20–30 Zeilen deiner `footer.html` posten? Dann kann ich sofort sagen, ob die Library fehlt oder ob sie nur falsch platziert ist.



```
{# Load navigation_tags at the top of the file: #}
{% load navigation_tags %}

<footer>
  <p>Built with Wagtail</p>

  {% with
    linkedin_url=settings.base.NavigationSettings.linkedin_url
    github_url=settings.base.NavigationSettings.github_url
    mastodon_url=settings.base.NavigationSettings.mastodon
    _url %}

  Show more ▾
```

In deinem Template wird tatsächlich ein Custom Template Tag verwendet:

```
{% load navigation_tags %}

django
```

Der Fehler entsteht also nicht durch `{% load navigation_tags %}` allein, sondern weil Django die zugehörige Tag-Library nicht finden kann.

Du brauchst irgendwo eine Datei:

```
base/
├── templatetags/
│   ├── __init__.py
│   └── navigation_tags.py
```

mit ungefähr folgendem Inhalt:

```
from django import template

register = template.Library()

NavigationLink = ...

Python
```

Da du außerdem verwendest:

`django`

vermute ich, dass du einem Wagtail-Tutorial folgst, das eine `NavigationSettings` -Klasse und eine `navigation_tags.py` definiert hat. Wahrscheinlich fehlt diese Datei oder der Ordner `templatetags`.

Prüfe als erstes

Existiert bei dir eine Datei:

```
base/templatetags/navigation_tags.py
```

oder

```
mysite/templatetags/navigation_tags.py
```

?

Falls nicht

Dann hast du wahrscheinlich einen Schritt aus dem Tutorial übersprungen. Oft sieht die Datei so aus:

```
from django import template
from wagtail.models import Site

register = template.Library()
```

Python

Falls du den Footer-Text gar nicht brauchst

Dann kannst du vorübergehend einfach beides entfernen:

`django`

und

django

Danach sollte die Seite wieder rendern.

Wenn du mir die Struktur deines Projekts zeigst (insbesondere die Apps wie `base`, `home`, `blog` usw.), kann ich dir genau sagen, wo `navigation_tags.py` hingehört und wie sie für deine `NavigationSettings` aussehen sollte.



ich habe die Datei `template_tags/navigation_tags.py`

Dann gibt es sehr wahrscheinlich ein Problem mit dem Pfad oder der App-Struktur.

Django findet Template Tags **nur** in einem Ordner namens:

```
<app_name>/
  templatetags/
    __init__.py
    navigation_tags.py
```

Wichtig:

✗ Falsch:

```
template_tags/
navigation_tags.py
```

oder

```
templates_tags/
navigation_tags.py
```

✓ Richtig:

```
base/
├── templatetags/
│   ├── __init__.py
│   └── navigation_tags.py
```

Außerdem müssen folgende Bedingungen erfüllt sein:

1. Der Ordner heißt exakt `templatetags` (ohne Unterstrich).
2. Im Ordner liegt eine Datei `__init__.py`.
3. Die zugehörige App (`base` , `blog` , etc.) steht in `INSTALLED_APPS`.
4. Nach Änderungen wurde der Django-Server neu gestartet.

Zeig bitte einmal:

```
project/  
└─ ...
```

bzw. die Ausgabe von:

Bash

oder zumindest:

- Wo genau liegt `navigation_tags.py` ?
- Wie heißt die App, in der sie liegt?
- Ist dort eine `__init__.py` im `templatetags` -Ordner vorhanden?

Dann lässt sich der Fehler meist sofort erkennen.



alles gemacht

Wenn 1–5 wirklich passen, dann gibt es noch ein paar weniger offensichtliche Ursachen.

Prüfe, ob Django die Library überhaupt sieht

Führe aus:

```
Bash
```

Dann:

```
from django.template import engines
```

```
Python
```

oder gezielter:

```
Python
```

Wenn `False` zurückkommt, wird die Library gar nicht registriert.

Zeige den exakten Pfad

Du schreibst:

```
ich habe die Datei template_tags/navigation_tags.py
```

Mich interessiert der **vollständige Pfad**, z. B.:

```
mysite/  
├─ base/  
│   └─ templatetags/  
│       └─ __init__.py  
│       └─ navigation_tags.py
```

oder

```
mysite/  
├─ mysite/  
│   └─ templatetags/  
│       └─ navigation_tags.py
```

Der zweite Fall ist z. B. problematisch, weil `mysite` oft keine Django-App ist.

Inhalt von `navigation_tags.py`

Selbst wenn der Import grundsätzlich klappt, kann schon eine einzelne fehlerhafte Zeile verhindern, dass die Library registriert wird.

Poste bitte den kompletten Inhalt von:

Python

Wagtail-Tutorial?

Der Ausschnitt

`django`

und

`django`

sieht stark nach dem offiziellen Wagtail-Tutorial aus.

Dort wird `navigation_tags.py` häufig in der App `base` angelegt. Wenn du die Datei in einer anderen App hast oder die App umbenannt hast, kann das ebenfalls die Ursache sein.

Ohne den tatsächlichen Pfad und den Inhalt von `navigation_tags.py` kann man jetzt nur raten. Wenn du beides postest, lässt sich der Fehler normalerweise direkt erkennen.



Traceback (most recent call last):
File "<console>", line 1, in <module>
NameError: name 'engines' is not defined

Das ist normal – du musst `engines` erst importieren.

In der Django-Shell:

```
from django.template import engines
```

Python

Noch hilfreicher ist:

```
from django.template import engines
```

Python

Dann siehst du alle registrierten Tag-Libraries.

Allerdings: Wenn du wirklich schon geprüft hast, dass

- der Ordner `templatetags` heißt,
- `__init__.py` vorhanden ist,
- die App in `INSTALLED_APPS` steht,
- `navigation_tags.py` existiert,
- `register = template.Library()` vorhanden ist,

dann ist der nächste Schritt nicht weiteres Raten, sondern den tatsächlichen Inhalt zu sehen.

Poste bitte:

1. Den kompletten Inhalt von `navigation_tags.py`
2. Den vollständigen Pfad der Datei (z. B. `base/templatetags/navigation_tags.py`)
3. Das Ergebnis von

Python

Dann lässt sich der Fehler gezielt eingrenzen. Oft steckt in `navigation_tags.py` ein Importfehler, den Django beim Laden der Tag-Library verschluckt und dann nur mit „not a registered tag library“ meldet.



dann ergibt sich true

Der Pfad ist `mysite/base/templatetags/navigation_tags.py`

Das ist ein sehr interessanter Hinweis:

Wenn

Python

True ergibt, dann ist die Tag-Library **registriert**. Das bedeutet:

- `templatetags` wird gefunden ✓
- `navigation_tags.py` wird geladen ✓
- die App ist in `INSTALLED_APPS` ✓

Dann passt die ursprüngliche Fehlermeldung wahrscheinlich nicht mehr zum aktuellen Stand des Projekts oder es gibt ein anderes Problem.

Ein paar Dinge zum Prüfen:

1. Server komplett neu starten

Nicht nur Auto-Reload.

Bash

Django cached Template-Libraries gelegentlich merkwürdig während der Entwicklung.

2. Wird wirklich dieselbe Django-Instanz verwendet?

Prüfe in der Shell:

```
Python
```

Es sollte genau auf

```
mysite/base/templatetags/navigation_tags.py
```

zeigen.

3. Ist die Fehlermeldung noch exakt dieselbe?

Wenn `navigation_tags` registriert ist, sollte dieser Fehler:

```
'navigation_tags' is not a registered tag library
```

eigentlich nicht mehr auftreten.

Dann wäre die nächste Frage:

Welche Fehlermeldung bekommst du jetzt exakt?

Bitte poste den vollständigen aktuellen Traceback.

4. Wagtail-Templates werden manchmal aus einem anderen Projekt geladen

Prüfe in `footer.html` testweise:

```
django
```

und entferne vorübergehend:

django

Wenn dann die Seite rendert, liegt das Problem nicht am Laden der Library, sondern an der Implementierung von `get_footer_text` .

Da `navigation_tags` laut Shell registriert ist, würde ich jetzt als Nächstes den Inhalt von `navigation_tags.py` sehen wollen. Besonders die Definition von:

```
@register.simple_tag
```

Python

Denn der Fehler dürfte inzwischen eher dort liegen als beim `{% load navigation_tags %}` .



ich habe 1. neu gestartet, 2. verweist genau darauf, passt also, 3. tritt noch genauso auf als Fehlermeldung

Das ist widersprüchlich:

- In der Django-Shell ist `navigation_tags` registriert (`True`).
- Der Importpfad zeigt auf die richtige Datei.
- Trotzdem sagt das Template-Rendering:

```
'navigation_tags' is not a registered tag library
```

Das deutet meist darauf hin, dass **die Shell und der laufende Server nicht dieselben Settings bzw. dieselbe Python-Umgebung verwenden.**

Prüfe im laufenden Projekt einmal:

Bash

Gibt es dort Fehler?

Und in der Shell:

Python

Ist dort deine `base`-App enthalten?

Außerdem würde ich gerne den Anfang deines `navigation_tags.py` sehen. Die ersten ~20 Zeilen reichen:

```
from django import template

register = template.Library()
```

Python

Noch eine gezielte Frage: Tritt der Fehler beim normalen Seitenaufruf auf oder im Wagtail-Admin?

Und poste bitte die **komplette Fehlermeldung inklusive der Zeilen direkt darüber**. Oft steht dort etwas wie:

```
TemplateSyntaxError at /
'navigation_tags' is not a registered tag library.
Must be one of:
...
```

und der Pfad des Templates. Das hilft einzugrenzen, ob eventuell ein anderes `footer.html` geladen wird als das, das du gerade bearbeitest.



python check gibt keine Fehler. base-App ist enthalten.

```
TemplateSyntaxError at /
'navigation_tags' is not a registered tag library. Must be
one of:
admin_list
admin_modify
admin_urls
```

auth
cache
i18n
Show more ▾

Die Datei ist grundsätzlich ok – aber hier ist der eigentliche Fehler sehr wahrscheinlich:

👉 Du importierst im Modul ein Model direkt:

Python

Wenn beim Import dieses Models *irgendetwas schiefgeht* (z. B. App-Registry noch nicht ready, circular import, DB-Zugriff, Wagtail-Sites/Settings Import etc.), dann passiert Folgendes:

- Django kann die Datei **nicht sauber als Template Library initialisieren**
- und fällt dann zurück auf:

```
'navigation_tags' is not a registered tag library
```

Das ist genau das typische Verhalten: **Importfehler wird „verschluckt“ und als Template-Fehler angezeigt.**

✅ **Fix (empfohlen): Import in die Funktion verschieben**

Mach das hier:

```

from django import template

register = template.Library()

@register.inclusion_tag("base/includes/footer_text.html", takes_context=True)
def get_footer_text(context):
    from base.models import FooterText # 🙌 WICHTIG: lazy import

    footer_text = context.get("footer_text", "")

    if not footer_text:
        instance = FooterText.objects.filter(live=True).first()
        footer_text = instance.body if instance else ""

Python
    }

```

🔥 Warum das Problem entsteht

Template tags werden beim Django-Start importiert.

Wenn dabei irgendein Modell:

- noch nicht vollständig geladen ist
- oder selbst etwas importiert, das zurück auf `templatetags` zeigt
- oder Wagtail/Settings initialisiert wird

🙌 dann bricht die Registrierung still ab.

💡 Warum Shell trotzdem „True“ sagt

Die Shell importiert Module oft in einem anderen Zustand (lazy / nach initialem App-Loading). Der Webserver lädt aber Templates früher im Startup-Flow → daher der Unterschied.

✅ Nach dem Fix

1. Server neu starten
2. Seite neu laden

Das Problem sollte weg sein.

Wenn du willst, kann ich dir auch zeigen, wie man solche Wagtail Footer/Settings sauberer über `SiteSettings` statt DB-Query im Template Tag baut (das ist die „Wagtail-Standardlösung“ und performanter).



`python manage.py migrate` sagt trotz Änderung der `navigation_tags.py` Seite "no changes detected". Ist das normal?

Ja — das ist **vollkommen normal**.

`migrate` hat mit deiner Änderung in `navigation_tags.py` **gar nichts zu tun**.

Warum „no changes detected“ erscheint

Bash

prüft nur:

- Models (`models.py`)
- Migrationen (`migrations/` Ordner)
- Datenbankstruktur

👉 **Template Tags gehören nicht dazu**

Deine Änderung betrifft nur Python-Code, nicht die DB

Du hast geändert:

Python

Das ist:

- Template-Logik
- kein Model

- kein Schema
- keine Migration nötig

Was du stattdessen tun musst

Nach Änderungen in `templatetags` gilt nur:

Bash

(oder Server neu starten)

Warum dein ursprünglicher Fehler wichtig war

Dein Problem war **kein Migration-Problem**, sondern:

- Template Tag wird beim Laden des Templates importiert
- dabei gab es sehr wahrscheinlich einen Import-/Initialisierungsfehler
- deshalb: „not a registered tag library“

Kurz gesagt

- ✓ `migrate` → irrelevant für Template Tags
- ✓ Änderung in `templatetags` → nur Server neu starten
- ✓ DB bleibt unberührt

Wenn du willst, kann ich dir kurz zeigen, wie man solche Footer-Daten in Wagtail „richtig sauber“ über `Site Settings` statt DB-Query im Template Tag löst (das ist der typische Wagtail-Weg).



den Fehler mit `navigation_tags` habe ich aber immer noch unbverändert

Dann ist jetzt ziemlich klar: Django lädt deine `navigation_tags` -Library im Webserver-Prozess nicht korrekt, obwohl sie importierbar ist. Das passiert fast immer aus einem von diesen zwei Gründen:

1. Du hast einen stillen Importfehler im Server-Startup (wird nur im Runserver-Kontext sichtbar)

Sehr typisch bei Wagtail/Django:

- 👉 Deine Library wird beim Start des Webserver importiert
- 👉 dabei knallt *irgendwo* ein Import
- 👉 Django markiert die Library dann als „nicht registriert“

Und genau deshalb siehst du nur:

```
'navigation_tags' is not a registered tag library
```

ohne echten Traceback.

🔥 Wichtigster Test (bitte genau so ausführen)

Stoppe den Server und starte ihn so:

```
Bash
```

Dann achte auf Importfehler beim Start.

🔥 Zweiter Test (der fast immer den echten Fehler zeigt)

```
Bash
```

Dann:

```
Python
```


👉 Wenn dort irgendein Fehler kommt, ist das die Ursache.

2. Sehr wahrscheinlicher echter Fehler in deinem Setup

Du hast:

Python

Das ist in Wagtail sehr oft problematisch, weil:

- `models.py` wiederum Settings / Pages / Wagtail-Sites importiert
- das kann zyklische Imports erzeugen
- oder `AppRegistryNotReady` im Startup-Kontext

👉 Ergebnis: Template Tag wird NICHT registriert

✅ FIX (robust & Wagtail-safe)

Mach den Import *lazy*:

```
from django import template

register = template.Library()

@register.inclusion_tag("base/includes/footer_text.html", takes_context=True)
def get_footer_text(context):
    from base.models import FooterText # 👉 wichtig

    footer_text = context.get("footer_text", "")

    if not footer_text:
        instance = FooterText.objects.filter(live=True).first()

    Python

    return {"footer_text": footer_text}
```

3. Der wichtigste Reality-Check

Bitte teste das jetzt sofort:

Bash

Dann:

Python

👉 Wenn DAS scheitert → Problem ist 100% Python Import (nicht Template)

🔴 4. Wenn es immer noch nicht geht

Dann gibt es nur noch eine echte Ursache:

👉 Du bearbeitest nicht die Datei, die Django tatsächlich lädt

Teste:

Python

✅ Fazit

Dein Problem ist nicht mehr:

- nicht registriert
- nicht gefunden
- nicht in INSTALLED_APPS

Sondern sehr wahrscheinlich:

👉 Import-Fehler beim Laden der Template Library (still verschluckt)

oder

👉 falscher Import-Kontext (zirkulärer Import durch models)

Wenn du mir noch 1 Sache gibst, kann ich es in einem Schritt fixen:

👉 Output von:

Python

oder Server-Start mit `--verbosity 3` (die ersten Fehlermeldungen).



1., 3., und 4. passt alles ohne Fehler#

Dann bleiben nur noch **zwei realistische Ursachen übrig** – und beide sind in Wagtail/Django sehr typisch:

Ursache 1: Du hast mehr als eine Django-Instanz / Settings-Umgebung

Das ist der Klassiker, wenn:

- Shell sagt „alles ok“
- Webserver sagt „tag library not registered“

 Dann laufen **Shell und Server mit unterschiedlichen Settings oder Python-Umgebungen**

Bitte prüf DAS im laufenden Server-Kontext

Erstelle kurz eine View oder nutze `shell_plus` :

Python

Und im Terminal:

Bash

 Wenn das nicht exakt gleich ist wie beim Runserver - Ursache gefunden

🔪 Ursache 2 (sehr wahrscheinlich): Django cached die Template-Library-Liste beim Start

Und deine Library wird zwar **importiert**, aber nicht in die aktive Engine übernommen.

Das passiert oft bei:

- Wagtail + autoreload Bugs
- alten `.pyc` Dateien
- oder falschem Reload-State

🔥 Fix (hart erzwingen)

Bitte einmal komplett:

```
Bash
```

Dann:

```
Bash
```

👉 wichtig: `--noreload`

🔪 Ursache 3 (die ich fast vermute): falscher App-Label Import-Kontext

Du hast:

```
mysite/base/templatetags/navigation_tags.py
```

Wenn deine App z. B. so registriert ist:

```
Python
```

aber irgendwo Templates laufen unter einem anderen App Label (z. B. `mysite.base` vs `base`), dann kann Django:

- 👉 Library importieren
- 👉 aber unter falschem namespace registrieren
- 👉 und `{% load navigation_tags %}` findet sie nicht

🔥 DER schnellste Beweis-Test

Füge ganz oben in `navigation_tags.py` ein:

Python

Dann:

Bash

Interpretation:

- ❌ du siehst nichts → Datei wird NICHT geladen → Pfad/App-Problem
- ✅ du siehst es → Import ok → Registrierung/Namespaces Problem

💡 Mein realistischer Verdacht (nach allem was du gezeigt hast)

Zu 80–90% ist es eins davon:

- 👉 falsche App-Label / Settings-Instanz
- 👉 oder Server läuft in anderer Umgebung als Shell
- 👉 oder App wird beim Wagtail startup anders geladen als erwartet

Wenn du mir EINE Sache gibst, kann ich es sofort final fixen:

Bitte poste:

Python

und den Output von:

Python

Dann ist das Problem in der Regel in einem Schritt gelöst.



which : Die Benennung "which" wurde nicht als Name eines Cmdlet, einer Funktion, einer Skriptdatei oder eines ausführbaren Programms erkannt.
Überprüfen Sie die Schreibweise des Namens, oder ob der Pfad korrekt ist (sofern enthalten), und wiederholen Sie den Vorgang.
In Zeile:1 Zeichen:1
+ which python
+ ~~~~~
+ CategoryInfo : ObjectNotFound: (which:String) [],
CommandNotFoundException
Show more ▾

Ah — das ist schon der entscheidende Kontext: du bist auf **Windows (PowerShell)**. `which` ist Linux/macOS, deshalb geht das bei dir nicht.

Windows-Äquivalente zu `which`

In PowerShell:

PowerShell

oder:

PowerShell



Warum das für dein Problem wichtig ist

Wir müssen jetzt wirklich prüfen, ob dein:

- runserver
- und deine Shell

👉 die gleiche Python-Installation benutzen

Bitte führe diese 2 Befehle aus:

1. Python im runserver-Kontext prüfen

Im Projektordner:

PowerShell

2. Dasselbe nochmal in Django:

PowerShell

Dann in der Shell:

Python



Wenn die Pfade unterschiedlich sind

Dann haben wir die Ursache für dein Problem:

👉 Shell und Server laufen in **verschiedenen Environments** (venv / global / WSL / etc.)

Und genau das führt zu:

```
navigation_tags is not a registered tag library
```

trotz korrekt registrierter Library.

Warum das dein Fehlerbild erklärt

Du hast bisher gesehen:

- Shell sagt: True ✓
- Server sagt: Library existiert nicht ✗

Das passiert **fast nie in Django selbst**, sondern fast immer durch:

unterschiedliche Python/venv Kontexte

Wenn das bestätigt ist (sehr wahrscheinlich)

Fix ist dann einfach:

PowerShell

```
.\venv\Scripts\activate  
python manage.py runserver
```

oder VS Code Python Interpreter korrigieren.

👉 Nächster Schritt

Schick mir bitte die beiden `sys.executable` Outputs.

Dann kann ich dir exakt sagen, wo der Bruch in deinem Setup ist (das ist hier sehr wahrscheinlich der eigentliche Root Cause).



where python ergibt nichts - aber bisher konnte ich die Site ja laden

Gut, das ist ein wichtiger Hinweis.

Wenn `where python` nichts zurückgibt, aber `python manage.py runserver` funktioniert, dann heißt das meistens:

👉 Windows findet Python nur über **App Execution Alias** oder über eine aktive Shell-Umgebung (z. B. VS Code / venv / WSL-Integration)

Das ist erstmal nicht der direkte Fehler.

🔥 Jetzt zum eigentlichen Problem (wichtig)

Du hast jetzt ein Setup, bei dem:

- `navigation_tags.py` existiert ✓
- `templatetags` Struktur korrekt ✓
- App in `INSTALLED_APPS` ✓
- Shell findet Library ✓
- Server sagt trotzdem: *not a registered tag library* ✗

👉 Das passt nicht mehr zu einem normalen Django-Problem

🔧 Sehr wahrscheinliche echte Ursache

👉 Du hast 2 verschiedene Django-Engines im Spiel

Typisches Szenario bei Wagtail:

- Haupt-Templates laufen über **Wagtail template engine**
- deine Shell testet **Django default engine**
- oder ein zweites Settings-Modul wird geladen

🔍 Der entscheidende Test (bitte machen)

Im laufenden Server:

Python

```
from django.template import engines

print(engines.all())
```

und:

Python

```
print(engines['django'].engine.template_libraries.keys())
```

Zweite sehr häufige Ursache bei genau deinem Fehlerbild

 `navigation_tags` wird in falscher App-Registry geladen

Das passiert, wenn:

- App heißt `base`
- aber Django erkennt sie als `mysite.base` oder umgekehrt
- Template Engine lädt andere App-Konfiguration

QUICK FIX TEST (sehr aufschlussreich)

Ändere in deinem Template:

django

```
{% load wagtailcore_tags %}
{% load navigation_tags %}
```

ZU:

django

```
{% load navigation_tags wagtailcore_tags %}
```

 und dann server neu starten

 Wenn du es sofort eingrenzen willst

Bitte führe das aus:

Bash

```
python manage.py shell
```

Dann:

Python

```
from django.template.loader import get_template
get_template("includes/footer.html")
```

👉 Wenn das fehlschlägt oder andere Fehler zeigt → falsches Template wird geladen

💡 Ehrliche Einschätzung

Nach allem was du gezeigt hast, sind die Top 2 Ursachen:

- 1️⃣ 1. falsche Template-Engine / App-Registry mismatch
- 2️⃣ 2. Django lädt andere App-Config als die Shell testet

👉 Was ich jetzt von dir brauche (damit wir es in 1 Schritt fixen)

Bitte poste:

1. Ausgabe von:

Python

```
from django.template import engines
print(engines.all())
```

2. Inhalt von:

Python

```
INSTALLED_APPS
```

Dann kann ich dir ziemlich sicher sagen, warum genau diese Library in der Web-Engine „verschwindet“.



wo soll ich den quickfix test machen? In footer.html steht ja nur load navigation_tags

Guter Punkt — der „Quickfix-Test“ war missverständlich formuliert.

👉 Den machst du **nicht irgendwo im Python-Code**, sondern **im Template** (footer.html) selbst.

🔧 So ist es korrekt

In deiner Datei:

```
mysite/base/templates/includes/footer.html
```

steht aktuell:

django

```
{% load navigation_tags %}
```

🔥 Quickfix-Test richtig gemacht

Ersetze das kurz testweise durch:

django

```
{% load wagtailcore_tags navigation_tags %}
```

Danach:

1. Server neu starten
2. Seite neu laden

Warum dieser Test sinnvoll ist

Wenn das plötzlich funktioniert, bedeutet das:

- 👉 Django erkennt die Library **nur nicht isoliert**
- 👉 aber sie wird geladen, sobald ein anderer Tag-Context aktiv ist

Das weist stark auf ein **Template Engine / App-Registry Timing-Problem** hin.

Wenn es NICHT funktioniert

Dann bleibt nur noch eine Ursache:

- 👉 `navigation_tags.py` wird **im Webprozess gar nicht importiert**, obwohl die Shell es kann

Dann ist der nächste Schritt nicht mehr Template-Code, sondern:

- falsches Template wird geladen
- oder falsche App wird zur Runtime verwendet

Mini-Check (sehr wichtig)

Füge ganz oben in `navigation_tags.py` ein:

Python

```
print("NAVIGATION_TAGS LOADED")
```

Dann:

- Server starten
- schauen ob es im Terminal erscheint

👉 Kurz gesagt

Den Quickfix machst du direkt im Template:

django

```
{% load wagtailcore_tags navigation_tags %}
```

Wenn du danach kurz sagst, was passiert (geht / geht nicht / Fehler bleibt gleich), kann ich dir den Fehler endgültig auf eine einzige Ursache eingrenzen.



Okay — dann sind wir jetzt in dem kleinen Kreis der wirklich „Django-typischen Edge Cases“.

Wenn:

- `templatetags/navigation_tags.py` wird importiert ✓
- `navigation_tags` ist in `template_libraries` ✓
- Settings stimmen ✓
- App-Struktur stimmt ✓
- Import im Server läuft ✓
- aber `{% load navigation_tags %}` wirft trotzdem *not registered* ✗

👉 dann bleibt nur noch ein sehr spezifischer Django-Mechanismus übrig:

🔧 Ursache: falsche Template-Engine zur Laufzeit

Django hat intern mehrere Engines:

- `django` (Standard)
- ggf. `wagtail` / eigene Engine-Wrapper
- ggf. `cached Loader`

👉 Und jetzt der entscheidende Punkt:

Deine Library wird in einer Engine registriert, aber das Template wird von einer anderen Engine gerendert.

🔍 DER 100%-Beweis (bitte jetzt ausführen)

Füge temporär in `navigation_tags.py` ein:

Python

```
from django.template import engines

print("ENGINES:", engines.all())
print("DJANGO LIBS:", engines['django'].engine.template_libraries.keys())
```

Dann Server starten und Seite laden.

Warum das wichtig ist

Wenn du z. B. siehst:

```
ENGINES: [<django>, <wagtail>]
```

dann ist der Fehler:

- 👉 {% load %} läuft über die Wagtail-Engine
- 👉 deine Library hängt aber an Django-Engine

QUICK FIX (Wagtail-spezifisch)

Falls das bestätigt wird, gibt es 2 Lösungen:

✅ Lösung A (sauber)

Stelle sicher, dass nur eine Engine Templates rendert:

Python

```
TEMPLATES = [  
    {  
        "BACKEND": "django.template.backends.django.DjangoTemplates",  
        "APP_DIRS": True,  
        ...  
    }  
]
```

- 👉 keine zweite Engine

⚠️ Lösung B (Workaround)

Explizit Library in beiden Engines registrieren (selten nötig)

Warum dein Fall „komisch“ wirkt

Weil du genau das typische Muster hast:

- ✓ alles ist korrekt
- ✓ import funktioniert
- ✗ aber Template sieht Library nicht

👉 Das ist kein App-Problem mehr, sondern Engine-Context mismatch

👉 Was ich jetzt brauche

Bitte poste nur diese 2 Outputs:

Python

```
from django.template import engines
print(engines.all())
```

und

Python

```
print(engines['django'].engine.template_libraries.keys())
```

Dann kann ich dir exakt sagen:

- 👉 welche Engine dein Template wirklich benutzt
- 👉 warum `navigation_tags` dort fehlt
- 👉 und der konkrete 1-Zeilen-Fix in deinem Setup

Wir sind jetzt sehr nah an der Ursache — das ist kein „Django rätselhaft“-Problem mehr, sondern nur noch Engine-Zuordnung.



ENGINES:

```
[<django.template.backends.django.DjangoTemplates
object at 0x0000028EF6196D50>]
```

```
DJANGO LIBS: dict_keys(['cache', 'i18n', 'l10n', 'static', 'tz',
'wagtailsettings_tags', 'wagtailembeds_tags',
'wagtailusers_tags', 'wagtailimages_tags',
'wagtailadmin_tags', 'wagtailuserbar', 'wagtail_cache',
'wagtailcore_tags', 'admin_list', 'admin_modify',
'admin_urls', 'log', 'auth'])
```

Jetzt haben wir den Treffer 🎯

Deine Ausgabe ist eindeutig:

```
DJANGO_LIBS:
...
wagtailcore_tags
wagtailimages_tags
...
(navigation_tags fehlt komplett)
```

👉 Das ist der entscheidende Beweis:

Deine `navigation_tags.py` wird zwar importiert, aber nicht in die Django `template_libraries` registriert

💣 Die eigentliche Ursache

Das passiert in Django nur in einem Fall:

👉 Die Datei liegt NICHT in einem „App Package“, das Django beim App-Loading für Template-Tags scannt

Und jetzt kommt der wichtige Punkt bei deinem Setup:

Du hast gesagt:

```
mysite/base/templatetags/navigation_tags.py
```

👉 Aber Django registriert Template Tags nur automatisch, wenn:

- ✓ die App wirklich als Python Package geladen wird
- ✓ und `templatetags` direkt darin liegt
- ✓ und die App importiert wird, bevor Templates geladen werden

🔥 Der echte Kernfehler (bei dir)

👉 Deine App `base` wird zwar importiert (du siehst den Print)

👉 aber sie ist nicht korrekt als App im App Registry „aktiv“ für Template Discovery

Das passiert oft bei Wagtail-Projekten, wenn:

❌ Fall 1: AppConfig wird nicht verwendet

In `INSTALLED_APPS` steht wahrscheinlich:

Python

```
'base'
```

statt:

Python

```
'base.apps.BaseConfig'
```

❌ Fall 2: App wird importiert, aber nicht „ready“

Django scannt Template Tags nur zuverlässig bei AppConfig-Load.

🚨 FIX (sehr wahrscheinlich deine Lösung)

👉 Öffne `settings/dev.py`

Und ändere:

❌ falls so:

Python

```
INSTALLED_APPS = [  
    'base',  
]
```

✅ auf:

Python

```
INSTALLED_APPS = [  
    'base.apps.BaseConfig',  
]
```


👉 und in `base/apps.py` sicherstellen:

Python

```
from django.apps import AppConfig

class BaseConfig(AppConfig):
    name = "base"
```

💡 Warum das genau dein Fehlerbild erklärt

- Import funktioniert ✔ (Python findet Modul)
- Print erscheint ✔ (Datei wird importiert)
- aber:
 - Template-Library Registry wird nicht aktualisiert ❌

👉 weil Django die App nicht über AppConfig initialisiert hat

💥 Warum das so „paradox“ aussieht

Django hat 2 Schritte:

1. Python importiert Modul → dein Print ✔
2. AppConfig registriert Template Tags → ❌ fehlt bei dir

🔧 Nach dem Fix

Danach:

Bash

```
CTRL+C
python manage.py runserver
```

📄 Erwartetes Ergebnis

Dann sollte erscheinen:

```
DJANGO LIBS:
...
```

```
navigation_tags
```

und dein Template:

```
django
```

```
{% load navigation_tags %}
```

funktioniert sofort.

👉 Kurzfassung

Dein Problem ist NICHT:

- Template Code
- Import
- Settings
- Engine

👉 sondern:

❌ App wird nicht über AppConfig korrekt registriert

Wenn du willst, kann ich dir danach noch zeigen, wie man das in Wagtail sauberer löst (Footer Settings ohne Custom Template Tags – ist eigentlich der Standardweg).



Es funktioniert!! Danke :)

Sehr gut — dann war es genau dieser App-Registry/AppConfig-Effekt 👍

Das ist ein klassischer Django/Wagtail-Fall: alles sieht korrekt aus, aber die Template-Library wird nicht in die aktive App-Initialisierung aufgenommen.

Wenn später nochmal sowas auftaucht, ist die Reihenfolge meist:

- AppConfig prüfen (BaseConfig statt nur 'base')
- INSTALLED_APPS Reihenfolge/Einträge checken
- Template-Tag Import „sichtbar, aber nicht registriert“ → fast immer App-Init Problem

Wenn du willst, kann ich dir als nächsten Schritt zeigen, wie man deinen Footer ohne Custom Template Tags sauber über Wagtail Settings (`SiteSettings`) löst — das ist im Alltag meist robuster und einfacher zu pflegen.

